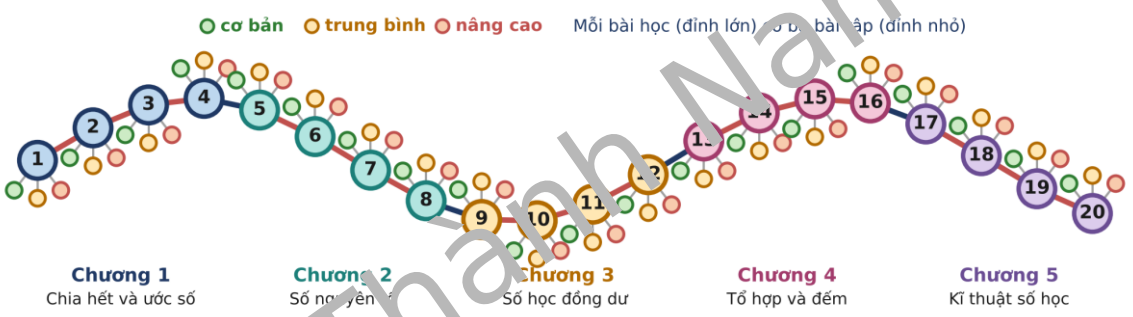
Tổng hợp và biên soạn: Lê Thành Nam

TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

SỐ HỌC

Từ cơ bản đến nâng cao

TẬP 3 · HƯỚNG DẪN GIẢI



Lê Thành Nam

Chương 1. CHIA HẾT VÀ ƯỚC SỐ

Chương 2. SỐ NGUYÊN TỐ

Chương 3. SỐ HỌC ĐỒNG DƯ

Chương 4. TỔ HỢP VÀ ĐẾM

Chương 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH

60 lời giải · từ cách ngây thơ đến cách tối ưu · chương trình đã chấm thật trên bộ test

LỜI NÓI ĐẦU

Tập 3 trình bày lời giải cho cả **60 bài tập** của Tập 2. Mỗi lời giải không chỉ đưa ra một chương trình đạt 100 điểm, mà đi lại đúng con đường một học sinh giỏi suy nghĩ trong phòng thi: bắt đầu từ cách ngây thơ, **đếm số phép tính** để thấy bế tắc, rồi phá bế tắc bằng một quan sát số học hay một công cụ phù hợp của Tập 1.

Số học trong lập trình có một đặc điểm riêng: ý tưởng thường gói gọn trong một công thức, nhưng khoảng cách giữa "đúng ý tưởng" và "đúng 100 điểm" lại nằm ở những chi tiết rất nhỏ: một phép nhân tràn số, một số dư âm, một trường hợp n = 1, một mô-đun không nguyên tố. Vì vậy, ngoài lời giải ngây thơ và lời giải tối ưu, nhiều bài còn có một lời giải **"trông như đúng"**: chương trình mà rất nhiều bạn sẽ nộp, qua được ví dụ nhưng mất điểm vì một cái bẫy. Em sẽ thấy rõ cái bẫy ấy làm mất bao nhiêu điểm.

Mọi chương trình trong sách đều đã được kiểm tra hai lần: đối chiếu với lời giải vét cạn (hoặc phép tính số lớn chính xác của Python) trên hàng trăm bộ dữ liệu nhỏ ngẫu nhiên, và chấm trên bộ test theo đúng các subtask của Tập 2, với giới hạn thời gian và bộ nhớ của từng bài. Các con số trong **bảng điểm** cuối mỗi bài là điểm thực tế mà từng chương trình nhận được, không phải ước lượng.

Lê Thành Nam

Lê Thành Nam

CẤU TRÚC MỘT LỜI GIẢI

Mục	Nội dung
1. Hiểu bài toán	Diễn đạt lại đề bằng ngôn ngữ số học, thường kèm một hình vẽ; nhận ra công cụ quen thuộc ẩn sau câu chuyện.
2. Cách ngây thơ	Sơ đồ khối, chương trình và độ phức tạp của cách làm đầu tiên ai cũng nghĩ ra.
3. Bế tắc	Cách hiện tại ăn được subtask nào, vì sao không đi xa hơn; đôi khi kèm một lời giải trung gian hoặc một lời giải sai "trông như đúng".
4. Phá bế tắc	Quan sát then chốt, công thức, hình vẽ chạy tay trên ví dụ, lập luận vì sao đúng.
5. Cách tối ưu	Sơ đồ khối, chương trình hoàn chỉnh và độ phức tạp.
6. Bẫy thường gặp	Những lỗi làm mất điểm hay gặp nhất: tràn số, số âm, trường hợp biên...
7. Bảng điểm	Điểm thực tế của từng cách làm trên bộ test. Dòng tô xanh là lời giải mẫu.

Các khung màu trong sách:

Khung	Ý nghĩa
ĐIỂM BẾ TẮC	Chỉ ra chính xác chỗ cách làm hiện tại "kẹt": phép tính nào lặp lại, đại lượng nào quá lớn.
VÌ SAO ĐÚNG? / CHỨNG MINH	Lập luận ngắn gọn cho một công thức hay một nhận xét then chốt.
MỞ RỘNG	Một cách làm khác, một bài toán tổng quát hơn, hoặc một mẹo cài đặt đáng nhớ.

Lê Thành Nam

> **LỜI KHUYÊN**
>
> Đừng đọc lời giải như đọc truyện. Sau mục 3, hãy gấp sách và tự tìm cách phá bế tắc trong ít nhất mười lăm phút. Sau mục 5, hãy tự viết lại chương trình mà không nhìn sách, rồi tự sinh test để đối chiếu với cách ngây thơ của chính em (Tập 2, Phụ lục B).
>
> Mỗi chương trình đọc từ tệp .INP nếu tệp tồn tại, ngược lại đọc từ bàn phím; vì vậy em có thể chạy thử trực tiếp hoặc đối chiếu bằng ống dẫn (<, >).

MỤC LỤC

LỜI NÓI ĐẦU **2**

CẤU TRÚC MỘT LỜI GIẢI **3**

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ **6**

Bài 1.1. NGÀY TRONG TUẦN 7
Bài 1.2. CHỮ SỐ CUỐI 12
Bài 1.3. VIẾT LIỀN 18
Bài 2.1. XẾP HÀNG 24
Bài 2.2. ƯỚC TĂNG DẦN 29
Bài 2.3. SỐ BẠN BÈ 34
Bài 3.1. CẮT GẠCH 40
Bài 3.2. BỎ MỘT SỐ 46
Bài 3.3. ĐÈN NHÁY 51
Bài 4.1. ĐONG NƯỚC 57
Bài 4.2. MUA HÀNG 62
Bài 4.3. ĐỒNG HỒ HỎNG 68

CHƯƠNG 2. SỐ NGUYÊN TỐ **74**

Bài 5.1. NGUYÊN TỐ GẦN NHẤT 75
Bài 5.2. TỔNG HAI SỐ NGUYÊN TỐ 80
Bài 5.3. NGUYÊN TỐ 64 BIT 85
Bài 6.1. ĐẾM SỐ NGUYÊN TỐ 91
Bài 6.2. SỐ NGUYÊN TỐ SINH ĐỐI 96
Bài 6.3. SỐ CÓ ĐÚNG BA ƯỚC 101
Bài 7.1. SỐ ƯỚC 106
Bài 7.2. CHỮ SỐ 9 CỦA GIAI THỪA 111
Bài 7.3. BCNN CỦA DÃY 116
Bài 8.1. NGUYÊN TỐ TRÊN ĐOẠN 122
Bài 8.2. KHOẢNG CÁCH LỚN NHẤT 128
Bài 8.3. SỐ ƯỚC TRÊN ĐOẠN 133

CHƯƠNG 3. SỐ HỌC ĐỒNG DƯ **138**

Bài 9.1. LŨY THỪA 139
Bài 9.2. SỐ MŨ KHỔNG LỒ 144
Bài 9.3. TỔNG LŨY THỪA 149
Bài 10.1. XÁC SUẤT 154
Bài 10.2. NGHỊCH ĐẢO DÃY 159
Bài 10.3. PHƯƠNG TRÌNH ĐỒNG DƯ 164
Bài 11.1. PHI 170
Bài 11.2. PHÂN SỐ TỐI GIẢN 175
Bài 11.3. THÁP LŨY THỪA 179
Bài 12.1. ĐIỂM BINH 185
Bài 12.2. HAI NGỌN ĐÈN 190
Bài 12.3. LỊCH CAN CHI 195

Lê Thành Nam

CHƯƠNG 4. TỔ HỢP VÀ ĐẾM 201
Bài 13.1. CHIA KẸO 202
Bài 13.2. ĐƯỜNG ĐI TRÊN LƯỚI 207
Bài 13.3. TỔ HỢP KHỔNG LỒ 212
Bài 14.1. BỘI SỐ 217
Bài 14.2. NGUYÊN TỐ CÙNG NHAU 222
Bài 14.3. TẶNG QUÀ 228
Bài 15.1. LÁT GẠCH 234
Bài 15.2. TRIBONACCI 239
Bài 15.3. ĐƯỜNG ĐI ĐỘ DÀI K 245
Bài 16.1. TỔNG ƯỚC 251
Bài 16.2. TỔNG DƯ 255
Bài 16.3. HAI THƯƠNG 259

CHƯƠNG 5. KĨ THUẬT SỐ HỌC TRONG LẬP TRÌNH 264
Bài 17.1. TỔNG LỚN 265
Bài 17.2. LŨY THỪA ĐẦY ĐỦ 270
Bài 17.3. TÍCH LỚN 274
Bài 18.1. ĐỔI CƠ SỐ 280
Bài 18.2. BIT TRONG ĐOẠN 285
Bài 18.3. SỐ CÓ K BIT 1 290
Bài 19.1. TỔNG BẰNG KHÔNG 295
Bài 19.2. ĐOẠN DÀI NHẤT 300
Bài 19.3. HÌNH CHỮ NHẬT CHIA HẾT 305
Bài 20.1. TRỒNG CÂY 310
Bài 20.2. ĐA GIÁC LƯỚI 315
Bài 20.3. TAM GIÁC NGUYÊN 320

PHỤ LỤC A. BẢNG TỔNG HỢP ĐIỂM 325

PHỤ LỤC B. THƯ VIỆN HÀM SỐ HỌC 327

Lê Thành Nam

CHƯƠNG 1. CHIA HẾT VÀ ƯỚC SỐ

Mười hai lời giải đầu tiên xoay quanh những kĩ năng nền móng: lấy số dư (kể cả số âm và số rất lớn), duyệt ước tới $\sqrt{n}$, thuật toán Euclid và Euclid mở rộng. Ý tưởng không khó; cái khó là đếm đúng số phép tính để biết cách ngây thơ dừng ở đâu, và giữ cho mọi đại lượng trung gian không tràn số.

Bài	Tên bài	Mức độ	Trọng tâm của lời giải
1.1	NGÀY TRONG TUẦN	★ CƠ BẢN	Lấy số dư không âm; tránh vòng lặp đếm từng ngày.
1.2	CHỮ SỐ CUỐI	★★ TRUNG BÌNH	Tìm chu kì của dãy số dư; xử lí số cho dưới dạng xâu.
1.3	VIẾT LIỀN	★★★ NÂNG CAO	Mở rộng sơ đồ Horner cho những "chữ số" có độ dài thay đổi; tiết kiệm bộ nhớ.
2.1	XẾP HÀNG	★ CƠ BẢN	Duyệt ước tới $\sqrt{n}$; xử lí trường hợp suy biến n = 1 và số nguyên tố.
2.2	ƯỚC TĂNG DẦN	★★ TRUNG BÌNH	Ghép hai dãy ước nhỏ, ước lớn thành dãy tăng; trả lời nhanh nhiều truy vấn trên dãy đã sắp.
2.3	SỐ BẠN BÈ	★★★ NÂNG CAO	Đổi chiều câu hỏi "k có những ước nào" thành "d là ước của những số nào"; tiết kiệm bộ nhớ.
3.1	CẮT GẠCH	★ CƠ BẢN	Dùng gcd cho số tới 10^{18}; nhận ra đáp số vượt long long và in số *int128*.
3.2	BỎ MỘT SỐ	★★ TRUNG BÌNH	Tính trước tiền tố, hậu tố để "bỏ một phần tử" trong O(1).
3.3	ĐÈN NHẤY	★★★ NÂNG CAO	Khai thác "chỉ có ít giá trị khác nhau" để trả lời truy vấn trên đoạn; BCNN không tràn số.
4.1	ĐONG NƯỚC	★ CƠ BẢN	Áp dụng Bézout cho nhiều số; trả lời truy vấn trong O(1).
4.2	MUA HÀNG	★★ TRUNG BÌNH	Tìm nghiệm có x nhỏ nhất không âm rồi đếm số nghiệm trong một đoạn; tránh tràn số.
4.3	ĐỒNG HỒ HỎNG	★★★ NÂNG CAO	Đưa bài toán về phương trình đồng dư; xử lí trường hợp suy biến và tràn số với mô-đun tới 10^{18}.

Lê Thành Nam

BÀI 1.1 NGÀY TRONG TUẦN

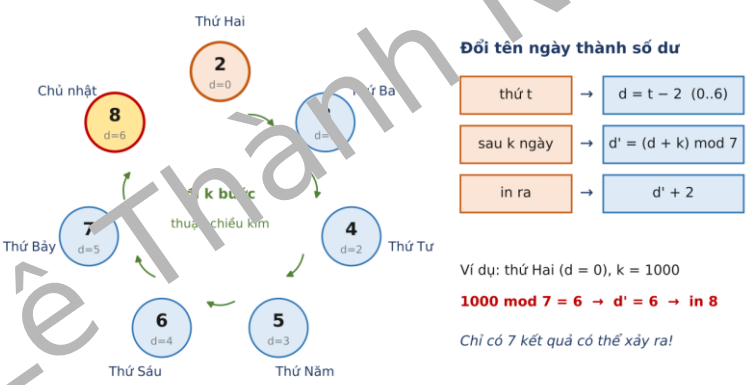
Dữ liệu vào	Kết quả	Thời gian	Bộ nhớ	Mức độ
NGAYTHU.INP	NGAYTHU.OUT	1 giây	256 MB	★ CƠ BẢN

Liên hệ: Tập 1, Bài 1 (Phép chia có dư và đồng dư) · Tập 2, bài tập 1.1

1. Hiểu bài toán

Mỗi câu hỏi cho "hôm nay là thứ t" và một số nguyên k, hỏi k ngày sau (hoặc |k| ngày trước) là thứ mấy. Các ngày trong tuần lặp lại theo vòng tròn 7 ngăn, nên bài toán thực chất là: **đi k bước trên một vòng tròn 7 ngăn** (k âm thì đi ngược chiều). Đây đúng là phép chia có dư cho 7 của Tập 1, Bài 1.

Mẹo đầu tiên là **đánh số lại cho đẹp**: các thứ 2, 3, ..., 8 không bắt đầu từ 0 nên rất khó làm việc với phép chia có dư. Đặt d = t − 2 thì thứ Hai ứng với d = 0, thứ Ba d = 1, ..., Chủ nhật d = 6: đúng bảy số dư có thể có khi chia cho 7.



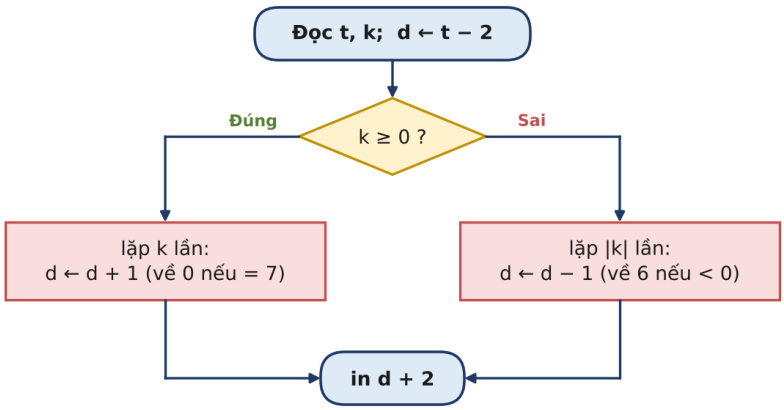
Hình 1.1.1. *Bảy ngày trên vòng tròn; mỗi ngày mang thêm "số dư" d = t − 2. Đi k bước theo chiều kim đồng hồ là cộng k rồi lấy dư cho 7.*

Ví dụ 1, câu hỏi thứ nhất: thứ Hai (d = 0), k = 1000. Vì 1000 = 7 · 142 + 6, đi 1000 bước là đi đúng 142 vòng trọn vẹn (quay về chỗ cũ) rồi thêm 6 bước: tới d = 6, tức Chủ nhật, in ra 8.

2. Cách ngây thơ: Đếm từng ngày

Cách tự nhiên nhất là mô phỏng đúng như ta lật lịch: tiến (hoặc lùi) từng ngày một, mỗi lần cập nhật d, khi d vượt quá 6 thì quay về 0 (khi d nhỏ hơn 0 thì quay về 6).

Lê Thành Nam



Hình 1.1.2. Sơ đồ khối của cách đếm từng ngày cho một câu hỏi

Cách ngây thơ: đếm từng ngày

```cpp
#include <cstdio>

int main() {
    if (fopen("NGAYTHU.INP", "r")) {
        freopen("NGAYTHU.INP", "r", stdin);
        freopen("NGAYTHU.OUT", "w", stdout);
    }
    int q;
    scanf("%d", &q);
    while (q--) {
        long long t, k;
        scanf("%lld %lld", &t, &k);
        int d = t - 2;                  // thứ Hai = 0, ..., Chủ nhật = 6
        if (k >= 0) {
            for (long long i = 0; i < k; i++) {     // tiến từng ngày
                d++;
                if (d == 7) d = 0;
            }
        } else {
            for (long long i = 0; i < -k; i++) {    // lùi từng ngày
                d--;
                if (d < 0) d = 6;
            }
        }
        printf("%d\n", d + 2);
    }
    return 0;
}
```

Lê Thành Nam

Độ phức tạp: O(|k|) cho mỗi câu hỏi, tổng cộng O(Q · max|k|). Với $Q = 10^5$ và $|k| = 10^6$ đã là 10^{11} bước; với $|k| = 10^{18}$ thì một câu hỏi thôi cũng cần 10^{18} bước, tức chạy khoảng ba mươi năm.

3. Bế tắc

Subtask 1 ($0 \le k \le 10^6$): nghe như dễ, nhưng Q vẫn tới 10^5. Bộ test có các test $Q = 10^5$, $k \approx 10^6$: khoảng 10^{11} bước, chạy quá giới hạn: **0 điểm**.

Subtask 2, 3 (k tới 10^{18}): không bao giờ chạy xong: **0 điểm**.

Một học sinh khác nhận ra ngay cần chia lấy dư và viết một dòng: `printf("%lld\n", (t - 2 + k) % 7 + 2);`. Chương trình chạy tức thì, đúng hết hai subtask đầu ($k \geq 0$) nhưng **sai** ở subtask 3: với $t = 2$, $k = -100$ nó in ra 0, một "thứ" không tồn tại. Tổng cộng **60 điểm**.

> **ĐIỂM BẾ TẮC**
>
> Có tới 10^{18} ngày cần "đi qua", nhưng chỉ có **7 kết quả** có thể xảy ra. Đếm từng ngày là làm đi làm lại cùng một vòng tròn.
>
> Còn khi dùng phép %, toán tử % của C++ **không** cho số dư theo nghĩa toán học khi số bị chia âm.

4. Phá bế tắc: Phép chia có dư

Bước 1. Bỏ các vòng tròn trọn vẹn

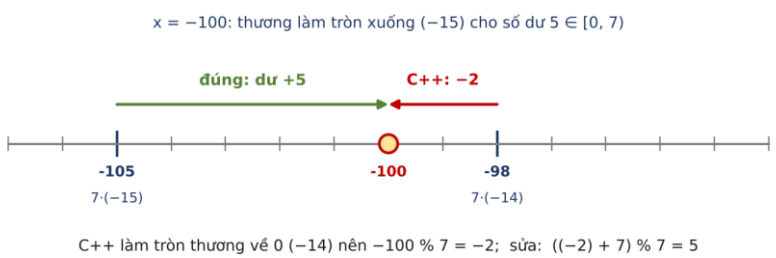
Đi 7 bước trên vòng 7 ngăn là quay về chỗ cũ. Vậy đi k bước cũng như đi r bước, với r là số dư của k khi chia cho 7. Tổng quát hơn, vị trí sau khi đi là số dư của d + k khi chia cho 7:

$$\mathbf{d' = (d + k) \bmod 7, \quad đáp\ số = d' + 1, \quad với\ d = t - 2.}$$

> **VÌ SAO ĐÚNG?**
>
> Viết $k = 7q + r$ với $0 \leq r < 7$ (q có thể âm khi k âm). Đi 7q bước là đi $|q|$ vòng trọn (xuôi hoặc ngược chiều), không thay đổi vị trí. Phần còn lại là r bước xuôi chiều. Hai số có cùng số dư khi chia cho 7 thì chỉ cùng một ngày: đó chính là đồng dư theo mô-đun 7 (Tập 1, Bài 1).
>
> Đặc biệt, **đi lùi** $|k|$ **ngày** cũng là đi xuôi một số bước $r \in [0, 7)$: lùi 1 ngày = tiến 6 ngày, lùi 100 ngày = tiến 5 ngày.

Bước 2. Lấy số dư không âm trong C++

Trong toán học, số dư của −100 khi chia cho 7 là 5, vì $-100 = 7 \cdot (-15) + 5$. Nhưng C++ làm tròn thương **về phía 0** (thương là −14), nên `-100 % 7` bằng −2: kết quả mang dấu của số bị chia.



Hình 1.1.3. Trên trục số, −100 nằm giữa hai bội của 7 là −105 và −98. Số dư toán học đo từ bội bên trái (5); C++ đo từ bội gần 0 hơn (−2).

Le Thanh Nam

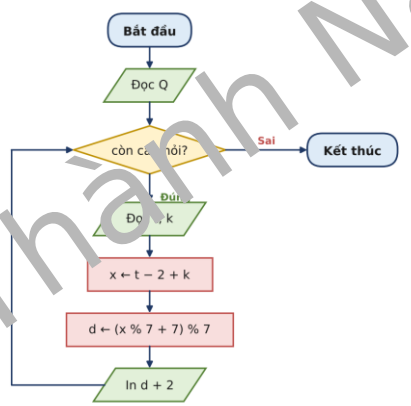
Cách sửa quen thuộc: `x % 7` luôn nằm trong (−7, 7), nên cộng thêm 7 được một số trong (0, 14), lấy dư thêm một lần nữa là về [0, 7): `((x % 7) + 7) % 7`

Chạy tay các ví dụ

t	k	x = t − 2 + k	x % 7 (C++)	((x % 7) + 7) % 7	In ra
2	1000	1000	6	6	**8**
2	−100	−100	−2	5	**7**
2	10^{18}	10^{18}	1	1	**3**
8	0	6	6	6	**8**
5	−7	−4	−4	3	**5**

Dòng thứ hai chính là chỗ cách "một dòng" ở mục 3 sai: nó in −2 + 2 = 0. Dòng cuối là một bẫy khác: x = −4 âm dù k = −7 chia hết cho 7.

5. Cách tối ưu: O(1) mỗi câu hỏi



Hình 1.1.4. Sơ đồ khối của lời giải tối ưu

Lê Thành Nam

NGAYTHU.CPP: số dư không âm, O(1) mỗi câu hỏi

```
#include <cstdio>

int main() {
    if (fopen("NGAYTHU.INP", "r")) {
        freopen("NGAYTHU.INP", "r", stdin);
        freopen("NGAYTHU.OUT", "w", stdout);
    }
    int q;
    scanf("%d", &q);
    while (q--) {
        long long t, k;
        scanf("%lld %lld", &t, &k);
        long long x = t - 2 + k;          // |x| <= 10^18 + 6: vừa long long
        long long d = (x % 7 + 7) % 7;    // số dư không âm, 0..6
        printf("%lld\n", d + 2);
    }
    return 0;
}
```

Độ phức tạp: O(1) cho mỗi câu hỏi, O(Q) cho cả bài; bộ nhớ O(1). Phần lớn thời gian là đọc và in 10^5 dòng, nên dùng scanf/printf.

> **MỞ RỘNG: KHÔNG CẦN PHÉP % THỨ HAI**
>
> Có thể viết `long long d = x % 7; if (d < 0) d += 7;`. Cách này dễ đọc và nhanh hơn một chút. Khi lập trình, nên viết sẵn một hàm `mod(x, m)` trả về số dư không âm và dùng lại ở mọi bài của chương.

6. Bẫy thường gặp

Quên số âm: `(t - 2 + k) % 7 + 2` cho kết quả 0, −1, ..., −4 khi $t - 2 + k < 0$. Mất trọn subtask 3.

"Cộng cho chắc dương": viết `(t - 2 + k + 7 * 1000000000000000000LL) % 7`: số $7 \cdot 10^{18}$ đã vượt long long ($\approx 9{,}2 \cdot 10^{18}$), tràn số. Nếu muốn cộng một bội của 7, chỉ cộng 7 sau khi đã lấy % 7.

Dùng int: k tới 10^{18} phải đọc bằng `%lld` vào `long long`.

Quên đánh số lại: lấy `t % 7` trực tiếp thì Chủ nhật (8) cho 1, thứ Bảy (7) cho 0: rất dễ nhầm. Đưa về $d = t - 2$ trước rồi mới tính.

7. Bảng điểm theo thuật toán

Thuật toán	Subtask 1 (30)	Subtask 2 (30)	Subtask 3 (40)	Tổng
Đếm từng ngày	0	0	0	0
(t − 2 + k) % 7, quên số âm	30	30	0	60
Số dư không âm, O(1) mỗi câu	30	30	40	**100**

Lê Thành Nam